

TOPIC

LLVM Pass Transformation Analyzer

Name: Vian Mangal

Register Number: 24BAI0134

Subject: Compiler Design Lab

Abstract

This project is a simple educational tool that shows how LLVM optimization passes change a C program. It collects LLVM IR snapshots, removes repeated output, and produces a short transformation report for students.

1. Introduction

Modern compilers perform many internal optimization steps before producing machine code. These steps are difficult to observe in normal compilation. **Motivation:** making the intermediate stages visible helps students learn and explain optimization. **Scope:** the first version supports small C files and a local Clang compiler. It does not execute input programs, modify LLVM, use AI, require a GPU, or need a server or database.

2. Problem Statement

LLVM can print the result of individual optimization passes, but the raw output is long and difficult to follow. A compact analyzer is needed to organize the output and show only the passes that meaningfully changed the program.

3. Objectives

- Accept one small C source file and run Clang safely.
- Extract the pass name, scope, order, and LLVM IR.
- Remove consecutive unchanged snapshots.
- Generate a readable timeline, manifest, and numbered IR files.
- Provide clear errors for invalid input or compilation failure.

4. Background Study

Clang converts C source into LLVM Intermediate Representation. LLVM then runs an ordered pass pipeline. Passes may simplify instructions, remove dead code, improve loops, or clean control flow. LLVM can print IR after each pass, but the dump must be parsed and organized before it is useful for learning.

5. System Design

C Source -> Clang -> IR Parser -> Change Filter -> Report

- Clang Runner: validates input and captures LLVM pass output.
- IR Parser: converts dump blocks into structured snapshots.
- Change Filter: removes consecutive unchanged snapshots per scope.
- Report Writer: produces JSON, Markdown, and .ll files.

Data flow and workflow: the C path enters the runner, raw pass output moves to the parser, filtered snapshots move to the report writer, and the user opens the generated timeline.

6. Methodology

- Check the source file and locate Clang.
- Run Clang at optimization level O1 without using a shell.
- Parse each IR dump and compare snapshots within the same scope.
- Retain meaningful changes and write the final report.
- Verify each module using automated tests and a real Clang smoke test.

7. Compiler Design Concepts Used

- Intermediate code: LLVM IR represents the program between source and machine code.
- Optimization passes: ordered stages analyze or transform the IR.
- Parsing: pass banners and IR blocks are converted into structured data.
- Function and module scope: unrelated snapshots are not compared.
- Dead-code elimination and simplification: common transformations visible in reports.

8. Implementation

Major modules cover the data model, parser, change filter, compiler runner, command-line interface, and report writer. The algorithm parses ordered snapshots, compares them by scope, and retains only meaningful changes.

Technology Stack

Python 3.10 or newer and its standard library are used for the analyzer and tests. Clang and LLVM provide the compiler pipeline. The interface is command-line based, and output is stored as JSON, Markdown, and LLVM .ll files.

9. Testing and Results

- Parser tests check valid, repeated, and malformed pass dumps.
- Compiler tests check discovery, safe command creation, and failure messages.
- Expected result: ordered snapshots with unchanged entries removed.
- Current result: snapshot parsing, filtering, and the Clang boundary are complete; 21 automated tests pass.
- Pending result: command-line and final report integration will complete the initial version.

10. Conclusion

The proposed analyzer makes a real LLVM optimization pipeline easier to understand. Its narrow C input scope and local design keep it technically useful, safe, and achievable for a student project.

11. Future Enhancements

- Show line-by-line differences between adjacent IR snapshots.
- Add simple descriptions for common optimization passes.
- Provide filters by function or pass name.
- Add a small local visual timeline after the command-line version is complete.

12. References

- LLVM Project, LLVM Language Reference Manual.
- LLVM Project, Using the New Pass Manager.
- Clang Project, Clang Compiler User's Manual.
- Keith Cooper and Linda Torczon, Engineering a Compiler.

13. Appendix

Prototype command: `python -m analyzer analyze examples/loop.c`. Expected output files are `manifest.json`, `timeline.md`, and numbered LLVM IR snapshots inside a report directory.